

MUP

LAB 10: Simplified Intel 8254 Timer Implementation

Objective

The objective of this lab is to design, implement, and verify a simplified version of the Intel 8254 Programmable Interval Timer in Verilog. The implementation will focus exclusively on Mode 0 (Interrupt on Terminal Count) operation of the timer.

Introduction

The Intel 8254 is a programmable interval timer/counter designed for microprocessor-based systems. It contains three independent counters, each capable of operating in one of 6 different modes. In this lab, you will implement a simplified version with the following characteristics:

- Support for Mode 0 (Interrupt on Terminal Count) only
- Three 8-bit counters (instead of 16-bit)
- Data bus buffer, control register, and read/write logic
- **Write-only operation: Only writing to registers and counters is implemented. Reading is not supported.**

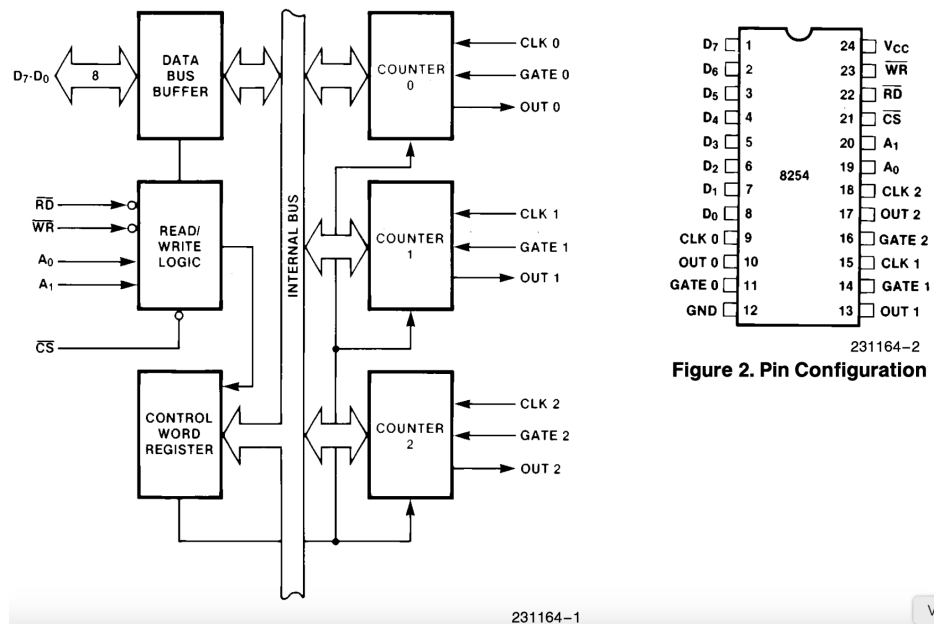


Figure 1: Simplified 8254 Block Diagram and Pin Configuration

Mode 0 Operation

In Mode 0 (Interrupt on Terminal Count), the operation is as follows:

- After the Control Word is written, OUT is initially low.
- OUT remains low until the counter reaches zero by being decremented after each clock cycle.
- When counter reaches zero, OUT goes high and remains high until a new count or new Control Word is written.
- GATE = 1 enables counting, GATE = 0 disables counting.

Module Descriptions

1. Data Bus Buffer

Handles data flow between the external data bus and internal modules. In this simplified implementation, only the write path is implemented (no read functionality).

Inputs: data_in[7:0]

Outputs: data_out[7:0]

2. Read/Write Logic

Decodes address inputs and generates write enable signals for counters and control register. Only write operations are handled.

Inputs: cs, wr, a1, a0

Outputs: ctr0_wr, ctr1_wr, ctr2_wr, ctrl_reg_wr

3. Control Register

Stores the control word for each counter. **All three control words are stored and continuously provided to their respective counters via wires.**

Inputs: clk, wr_en, data_in[7:0]

Outputs: ctr0_control[7:0], ctr1_control[7:0], ctr2_control[7:0]

4. Counter

Implements Mode 0 countdown with terminal count detection.

Inputs: clk, gate, ctr_wr, data_in[7:0], control_word[7:0]

Outputs: out

5. Top Module (To be completed by students)

The Top Module connects all submodules together to create a complete 8254 timer.

Top Module Template

```
1  'include "rw_logic.v"
2  'include "control_register.v"
3  'include "counter.v"
4
5  module intel_8254_simplified (
6      input  clk,
7      input  wr,
8      input  rd,
9      input  cs,
10     input  a1,
11     input  a0,
12     input  [7:0] data_in,
13     output [7:0] data_out,  // For data bus buffer
14     input  gate0,
15     input  gate1,
16     input  gate2,
17     output out0,
18     output out1,
19     output out2
20 );
21     // TO BE COMPLETED BY STUDENTS
22     // Declare internal signals
23
24     // Instantiate Read/Write Logic
25
26     // Instantiate Data Bus Buffer
27
28     // Instantiate Control Register
29
30     // Instantiate Counters
31
32 endmodule
```

Important Note

In this simplified implementation, **only write operations to the registers and counters are implemented**. Reading is not supported. All three control words are stored in the control register and provided to the counters via wires, with no control word write strobes.

Provided Code

The following modules are provided:

- `rw_logic.v` - Read/Write Logic Module
- `data_bus_buffer.v` - Data Bus Buffer Module
- `control_register.v` - Control Register Module
- `counter.v` - Counter Module

Assignment

1. Complete the top-level module (`intel_8254_simplified`) by connecting all submodules according to the block diagram.
2. Ensure your implementation passes all test cases in the provided test-bench.
3. Submit your completed Verilog file.

Marking Scheme

- Attendance: 1 mark
- Attempt at implementation: 3 marks
- Correct functionality (based strictly on test cases passed): 6 marks

Submission Requirements

- Complete Verilog code for the top module
- Simulation results showing all test cases pass
- Brief explanation of your implementation